

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Modelling Comparative Analysis
Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)*

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A
Modelling Comparative Analysis Assignment

Code of Conduct:

I Nikhil Kumar Reddy Kadiveti_ (192425031) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	
10. Documentation, GitHub Repository and Presentation	2%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Question / Problem Statement
1	Automobile Fault Diagnosis: An automobile service center wants to identify vehicle faults based on symptoms such as engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. Model the problem using production rules, propositional logic, first-order logic, and Prolog. Demonstrate forward and backward chaining and compare the approaches based on expressiveness, inference, complexity, and suitability.
2	Industrial Machine Fault Diagnosis: A manufacturing plant needs to identify machine faults using observations such as abnormal vibration, high temperature, unusual noise, pressure variation, and reduced production speed. Develop different knowledge representation models and compare their ability to represent machine components, relationships, rules, and diagnostic conclusions.
3	Cybersecurity Threat Detection: A security operations center wants to identify threats from repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Model the threat-detection problem using production rules, propositional logic, first-order logic, and Prolog, and compare forward and backward reasoning for deriving security conclusions.
4	Healthcare Diagnosis: A healthcare system needs to identify possible diseases from symptoms such as fever, cough, fatigue, breathing difficulty, and body pain. Construct comparative knowledge models using production rules, propositional logic, first-order logic, and Prolog. Analyze which representation provides better expressiveness and reasoning capability for diagnosis.
5	Agricultural Crop Disease Diagnosis: Farmers need assistance in identifying crop diseases based on leaf color, spots, wilting, soil conditions, humidity, and pest observations. Represent the agricultural knowledge using different modelling approaches and compare their effectiveness in representing relationships, applying rules, and deriving recommendations.
6	Banking Loan Eligibility: A bank wants to determine loan eligibility using income, credit score, employment status, existing loans, and repayment history. Model the decision-making process using production rules, propositional logic, first-order logic, and Prolog, and compare the reasoning capabilities of each approach.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Problem Statement & Objectives	Describe the selected real-world problem, domain, decision-making requirement, and objectives of the modelling exercise.
2	Domain Knowledge & Facts	Identify entities, attributes, facts, relationships, conditions, and conclusions relevant to the selected problem.
3	Production Rule Model	Represent the domain using IF–THEN production rules and explain how the rules derive conclusions.
4	Propositional Logic Model	Represent the same problem using propositions, logical connectives, and suitable inference statements.
5	First-Order Logic Model	Represent entities, relationships, properties, and rules using predicates and quantifiers.
6	Prolog Model & Implementation	Convert the knowledge into Prolog facts, rules, predicates, and queries. Include relevant code and outputs.
7	Forward & Backward Chaining	Demonstrate both reasoning mechanisms using selected facts/goals and provide the inference sequence.
8	Comparative Analysis (Table structure given)	Compare the models based on expressiveness, inference mechanism, complexity, scalability, flexibility, explainability, and suitability.
9	Results, Discussion & Recommended Model	Analyze the results and justify which approach is most appropriate for the selected real-world problem. Discuss limitations and possible improvements.
10	Conclusion, References & GitHub	Summarize findings, provide references, and include the GitHub repository link containing Prolog code, models, test cases, and supporting materials.

Compulsory Comparative Table:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation				
Expressiveness				
Rule Representation				
Inference Mechanism				
Forward Chaining				
Backward Chaining				
Handling Relationships				
Scalability				
Explainability				
Ease of Implementation				
Suitability for the Problem				

1. Problem Statement & Objectives

Automobile service centres routinely face the challenge of diagnosing vehicle faults quickly and accurately from a limited set of observable symptoms — engine overheating, starting failure, abnormal noise, low mileage (poor fuel efficiency), and dashboard warning indicators. Traditionally this diagnosis depends on the experience of a human mechanic, which introduces variability, delay, and the risk of missed or incorrect conclusions, particularly for junior technicians or high-volume service floors.

This assignment addresses the problem by constructing a rule-based expert system that encodes a mechanic's diagnostic knowledge as formal, machine-processable rules. The same body of domain knowledge is represented using four progressively more expressive modelling paradigms — production rules, propositional logic, first-order logic (FOL), and Prolog — so that their relative strengths for this diagnostic task can be directly compared.

1.1 Domain

The domain is automotive fault diagnosis at a service centre. The system reasons over a single vehicle at a time (though the FOL and Prolog models generalise to a fleet of vehicles), taking observed symptoms as input facts and deriving the most likely underlying fault together with a recommended corrective action.

1.2 Objectives

- Identify the domain entities, facts, and relationships relevant to automobile fault diagnosis.
- Construct a complete set of IF–THEN production rules mapping symptoms to faults and recommended actions.
- Formalise the same knowledge using propositional logic (connectives, implications) and first-order logic (predicates, variables, quantifiers).
- Implement an executable Prolog program with facts, rules and queries that reproduces the diagnostic behaviour.
- Demonstrate forward chaining (data-driven) and backward chaining (goal-driven) reasoning over the rule base.
- Compare all four approaches on expressiveness, inference mechanism, complexity, scalability, explainability and overall suitability, and recommend the best-fit approach for deployment.

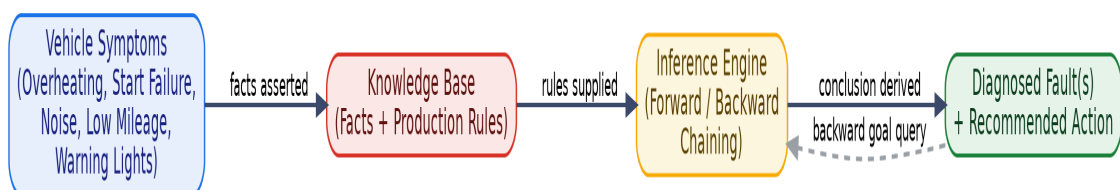


Figure 1: High-level architecture of the automobile fault-diagnosis expert system

2. Domain Knowledge & Facts

The knowledge base is built around two categories of entities: Symptoms, which are facts either observed directly by the technician or derived by intermediate rules, and Faults, which are the diagnostic conclusions the system aims to reach. Every entity is scoped to a Vehicle, allowing the same knowledge to be reused across many cars at the service centre.

2.1 Entities and Attributes

- Vehicle (V) — the car under diagnosis, identified by registration/ID (e.g., car1).
- Symptom (S) — an observable or derived condition of the vehicle (Table 1).
- Fault (F) — a diagnostic conclusion representing a faulty subsystem (Table 2).

- Recommendation (R) — the corrective action associated with a fault.

2.2 Symptom / Fact Inventory

ID	Symptom / Fact	Type
S1	coolant_level_low — coolant reservoir below minimum mark	Observed
S2	temp_gauge_high — temperature gauge reading in red zone	Observed
S3	engine_overheating — engine temperature exceeds safe limit	Derived
S4	battery_voltage_low — battery voltage below 11.8 V	Observed
S5	starter_clicks_no_start — starter clicks but engine does not turn over	Observed
S6	engine_fails_to_start — engine does not start on ignition	Derived
S7	knocking_sound — metallic knocking heard from engine bay	Observed
S8	belt_squealing — high-pitched squeal from accessory belt	Observed
S9	abnormal_noise_present — any unusual sound during operation	Derived
S10	high_fuel_consumption — fuel economy below rated average	Observed
S11	air_filter_clogged — air filter visibly obstructed	Observed
S12	low_mileage — poor kilometres-per-litre performance	Derived
S13	check_engine_light_on — MIL / check-engine light illuminated	Observed
S14	oil_pressure_warning_on — oil pressure warning lamp active	Observed
S15	warning_indicator_active — any dashboard warning lamp active	Derived

Table 1: Symptom / fact inventory used across all four models

2.3 Fault (Conclusion) Inventory

ID	Diagnosed Fault (Conclusion)
F1	cooling_system_failure — radiator / coolant / thermostat malfunction
F2	battery_starting_failure — battery, starter motor or alternator malfunction
F3	engine_mechanical_fault — internal engine wear or mechanical damage
F4	fuel_system_inefficiency — clogged filter / injector / carburation issue
F5	sensor_ecu_fault — sensor or ECU malfunction not explained by another fault

Table 2: Fault categories derived by the expert system

2.4 Relationships

- indicates(Symptom, Fault) — a symptom (or symptom combination) points toward a fault.
- causes(Fault, Recommendation) — a diagnosed fault triggers a specific corrective action.
- has_symptom(Vehicle, Symptom) — links an observed condition to a specific vehicle.
- has_fault(Vehicle, Fault) — links a diagnosed conclusion to a specific vehicle.

3. Production Rule Model

Production rules express domain knowledge as condition–action (IF–THEN) pairs. Each rule fires when its IF-part (antecedent) matches the current set of known facts, asserting its THEN-part (consequent) as a new fact. Ten diagnostic rules (R1–R10) map symptoms to faults, and five recommendation rules (R11–R15) map each fault to a corrective action — giving fifteen rules in total.

Rule	IF (Condition)	THEN (Action / Conclusion)
R1	coolant_level_low(V) AND temp_gauge_high(V)	engine_overheating(V)
R2	engine_overheating(V)	cooling_system_failure(V)
R3	battery_voltage_low(V) AND starter_clicks_no_start(V)	engine_fails_to_start(V)
R4	engine_fails_to_start(V)	battery_starting_failure(V)
R5	knocking_sound(V) OR belt_squealing(V)	abnormal_noise_present(V)
R6	abnormal_noise_present(V) AND knocking_sound(V)	engine_mechanical_fault(V)
R7	high_fuel_consumption(V) AND air_filter_clogged(V)	low_mileage(V)
R8	low_mileage(V)	fuel_system_inefficiency(V)
R9	check_engine_light_on(V) OR oil_pressure_warning_on(V)	warning_indicator_active(V)
R10	warning_indicator_active(V) AND NOT overheating(V) AND NOT fails_to_start(V)	sensor_ecu_fault(V)
R11	cooling_system_failure(V)	recommend(V, inspect_radiator_and_coolant)
R12	battery_starting_failure(V)	recommend(V, inspect_battery_and_starter)
R13	engine_mechanical_fault(V)	recommend(V, inspect_engine_internals)
R14	fuel_system_inefficiency(V)	recommend(V, replace_air_filter_service_fuel_system)
R15	sensor_ecu_fault(V)	recommend(V, run_obd_scan_check_sensors)

Table 3: Complete production rule set for automobile fault diagnosis

3.1 How Conclusions Are Derived

The rule engine repeatedly scans the rule base for a rule whose condition is fully satisfied by the working memory (the current set of known facts), fires it, and adds its conclusion as a new fact. This match–fire–assert cycle continues until no further rule can fire (forward chaining) or until a specific goal fact has been proved / disproved (backward chaining). For example, once coolant_level_low and temp_gauge_high are both known, R1 fires to add engine_overheating, which then satisfies R2 and produces cooling_system_failure — this chained firing is what allows the system to reach multi-step conclusions from raw symptoms.

4. Propositional Logic Model

Every symptom and fault is represented as an atomic proposition (true/false), and each production rule is rewritten as a logical implication built from the connectives AND ($\wedge$), OR ($\vee$), NOT ($\neg$) and IMPLIES ($\rightarrow$).

4.1 Propositions

Symbol	Proposition	Symbol	Proposition
p1	coolant_level_low	p11	abnormal_noise_present
p2	temp_gauge_high	p12	engine_mechanical_fault
p3	engine_overheating	p13	high_fuel_consumption
p4	cooling_system_failure	p14	air_filter_clogged
p5	battery_voltage_low	p15	low_mileage
p6	starter_clicks_no_start	p16	fuel_system_inefficiency
p7	engine_fails_to_start	p17	check_engine_light_on
p8	battery_starting_failure	p18	oil_pressure_warning_on
p9	knocking_sound	p19	warning_indicator_active
p10	belt_squealing	p20	sensor_ecu_fault

Table 4: Propositional symbols for the automobile fault-diagnosis domain

4.2 Logical Formulas

$(p1 \wedge p2) \rightarrow p3$ // coolant low & temp high $\rightarrow$ overheating
 $p3 \rightarrow p4$ // overheating $\rightarrow$ cooling system failure
 $(p5 \wedge p6) \rightarrow p7$ // low battery & starter clicks $\rightarrow$ fails to start
 $p7 \rightarrow p8$ // fails to start $\rightarrow$ battery / starting failure
 $(p9 \vee p10) \rightarrow p11$ // knock or squeal $\rightarrow$ abnormal noise present
 $(p11 \wedge p9) \rightarrow p12$ // abnormal noise + knock $\rightarrow$ mechanical fault
 $(p13 \wedge p14) \rightarrow p15$ // high consumption & clogged filter $\rightarrow$ low mileage
 $p15 \rightarrow p16$ // low mileage $\rightarrow$ fuel system inefficiency
 $(p17 \vee p18) \rightarrow p19$ // CEL or oil warning $\rightarrow$ warning indicator active
 $(p19 \wedge \neg p3 \wedge \neg p7) \rightarrow p20$ // warning active, no overheating/start fault $\rightarrow$ sensor/ECU fault

4.3 Sample Inference (Modus Ponens)

Given the facts $p1 = \text{TRUE}$ and $p2 = \text{TRUE}$ for a vehicle, Modus Ponens on the first formula lets us conclude $p3$ (engine_overheating). Applying Modus Ponens again on $p3 \rightarrow p4$ (a form of hypothetical syllogism when chained) lets us conclude $p4$ (cooling_system_failure):

1. $p1$ (given: coolant_level_low = TRUE)
2. $p2$ (given: temp_gauge_high = TRUE)
3. $p1 \wedge p2$ (conjunction, 1 & 2)
4. $(p1 \wedge p2) \rightarrow p3$ (rule R1 as implication)
5. $p3$ (Modus Ponens, 3 & 4) $\Rightarrow$ engine_overheating
6. $p3 \rightarrow p4$ (rule R2 as implication)
7. $p4$ (Modus Ponens, 5 & 6) $\Rightarrow$ cooling_system_failure

This derivation illustrates both the strength and the limitation of propositional logic: the reasoning steps are formally valid and easy to verify, but the model has no notion of "vehicle" as a variable — a separate, fully-duplicated set of twenty propositions (and rules) would be required for every additional car, which quickly becomes unmanageable at fleet scale.

5. First-Order Logic Model

First-order logic overcomes propositional logic's scalability problem by introducing variables, predicates, and quantifiers, so a single rule applies to every vehicle rather than being duplicated per vehicle.

5.1 Predicates

- CoolantLow(V), TempGaugeHigh(V), EngineOverheating(V), CoolingSystemFailure(V)
- BatteryVoltageLow(V), StarterClicksNoStart(V), EngineFailsToStart(V), BatteryStartingFailure(V)
- KnockingSound(V), BeltSquealing(V), AbnormalNoisePresent(V), EngineMechanicalFault(V)
- HighFuelConsumption(V), AirFilterClogged(V), LowMileage(V), FuelSystemInefficiency(V)
- CheckEngineLightOn(V), OilPressureWarningOn(V), WarningIndicatorActive(V), SensorECUFault(V)
- HasFault(V, F) — generic relation linking a vehicle V to a fault type F

5.2 Quantified Rules

```
∀V (CoolantLow(V) ∧ TempGaugeHigh(V) → EngineOverheating(V))
∀V (EngineOverheating(V) → CoolingSystemFailure(V))
∀V (BatteryVoltageLow(V) ∧ StarterClicksNoStart(V) → EngineFailsToStart(V))
∀V (EngineFailsToStart(V) → BatteryStartingFailure(V))
∀V (KnockingSound(V) ∨ BeltSquealing(V) → AbnormalNoisePresent(V))
∀V (AbnormalNoisePresent(V) ∧ KnockingSound(V) → EngineMechanicalFault(V))
∀V (HighFuelConsumption(V) ∧ AirFilterClogged(V) → LowMileage(V))
∀V (LowMileage(V) → FuelSystemInefficiency(V))
∀V (CheckEngineLightOn(V) ∨ OilPressureWarningOn(V) → WarningIndicatorActive(V))
∀V (WarningIndicatorActive(V) ∧ ¬EngineOverheating(V) ∧ ¬EngineFailsToStart(V)
    → SensorECUFault(V))
```

5.3 Ground Instance and Existential Query

Substituting the constant car1 for the variable V grounds the universal rule into a specific case:

```
CoolantLow(car1) ∧ TempGaugeHigh(car1) → EngineOverheating(car1)
```

An existential statement can also express service-centre-wide queries, such as "is there any vehicle currently in for service with a cooling system failure":

```
∃V ( HasFault(V, cooling_system_failure) )
```

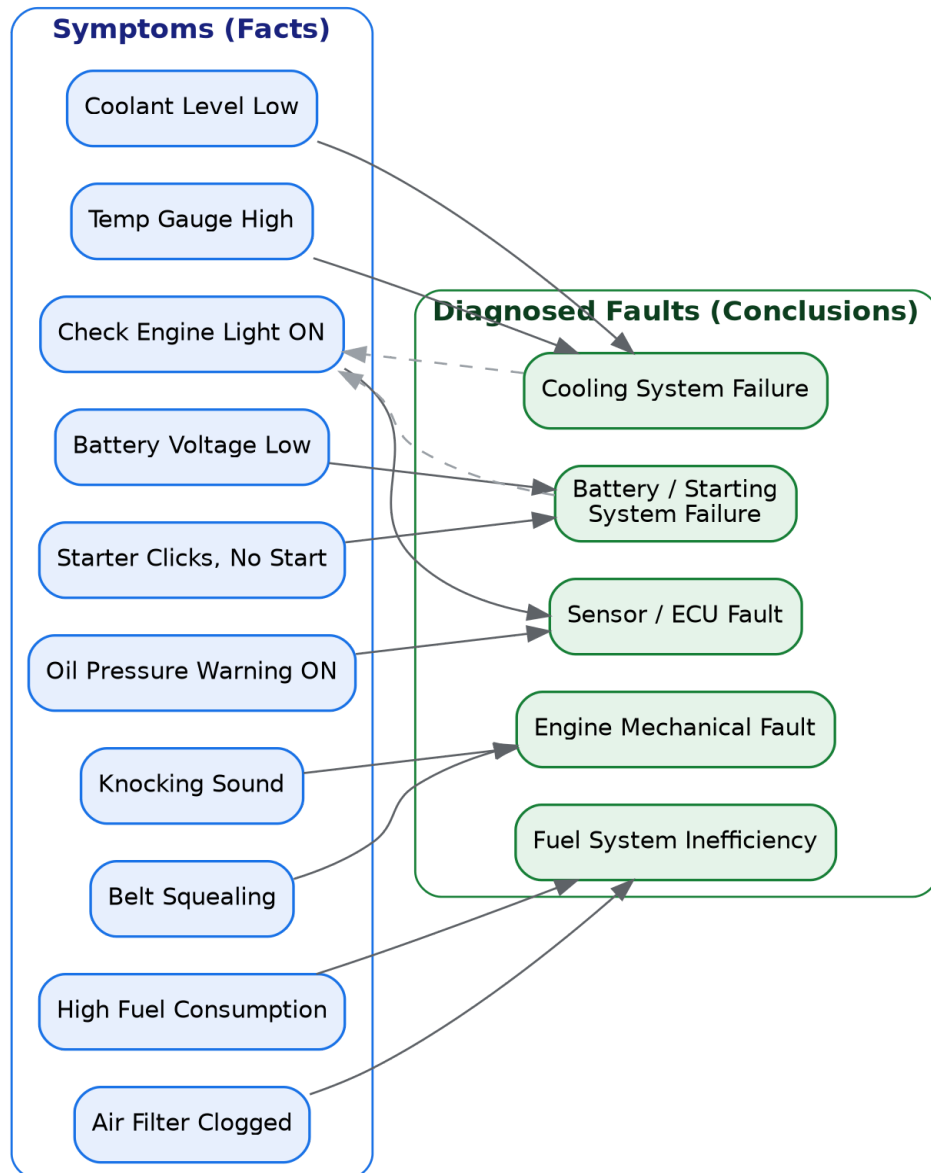


Figure 2: Symptom-to-fault relationship network — the same structure FOL predicates and Prolog clauses generalise over every vehicle V

6. Prolog Model & Implementation

The first-order rules translate almost directly into Prolog Horn clauses (head :- body), giving an executable version of the knowledge base. Facts represent observed symptoms per vehicle; rules derive faults and recommendations; queries drive both forward- and backward-style reasoning.

6.1 Facts and Rules

```
% ----- Facts: observed symptoms per vehicle -----
symptom(car1, coolant_level_low).
symptom(car1, temp_gauge_high).
symptom(car1, check_engine_light_on).

symptom(car2, battery_voltage_low).
symptom(car2, starter_clicks_no_start).

symptom(car3, knocking_sound).
symptom(car3, belt_squealing).

symptom(car4, high_fuel_consumption).
symptom(car4, air_filter_clogged).

% ----- Rules: diagnostic derivation -----
engine_overheating(V) :-
    symptom(V, coolant_level_low),
    symptom(V, temp_gauge_high).
cooling_system_failure(V) :- engine_overheating(V).

engine_fails_to_start(V) :-
    symptom(V, battery_voltage_low),
    symptom(V, starter_clicks_no_start).
battery_starting_failure(V) :- engine_fails_to_start(V).

abnormal_noise_present(V) :- symptom(V, knocking_sound).
abnormal_noise_present(V) :- symptom(V, belt_squealing).
engine_mechanical_fault(V) :-
    abnormal_noise_present(V),
    symptom(V, knocking_sound).

low_mileage(V) :-
    symptom(V, high_fuel_consumption),
    symptom(V, air_filter_clogged).
fuel_system_inefficiency(V) :- low_mileage(V).

warning_indicator_active(V) :- symptom(V, check_engine_light_on).
warning_indicator_active(V) :- symptom(V, oil_pressure_warning_on).
sensor_ecu_fault(V) :-
    warning_indicator_active(V),
    \+ engine_overheating(V),
    \+ engine_fails_to_start(V).

% ----- Rules: recommended action -----
recommend(V, inspect_radiator_and_coolant)    :- cooling_system_failure(V).
recommend(V, inspect_battery_and_starter)     :- battery_starting_failure(V).
recommend(V, inspect_engine_internals)        :- engine_mechanical_fault(V).
recommend(V, replace_air_filter_service_fuel_system) :- fuel_system_inefficiency(V).
recommend(V, run_obd_scan_check_sensors)      :- sensor_ecu_fault(V).

% ----- Top-level diagnosis predicate -----
diagnose(V, Fault, Recommendation) :-
    ( cooling_system_failure(V) -> Fault = cooling_system_failure
    ; battery_starting_failure(V) -> Fault = battery_starting_failure
    ; engine_mechanical_fault(V) -> Fault = engine_mechanical_fault
    ; fuel_system_inefficiency(V) -> Fault = fuel_system_inefficiency
    ; sensor_ecu_fault(V) -> Fault = sensor_ecu_fault
    ; Fault = no_fault_detected
    ),
    ( Fault == no_fault_detected -> Recommendation = none
```

```
; recommend(V, Recommendation) ).
```

6.2 Sample Queries and Outputs

```
?- cooling_system_failure(car1).
true.

?- diagnose(car1, Fault, Rec).
Fault = cooling_system_failure,
Rec = inspect_radiator_and_coolant.

?- diagnose(car2, Fault, Rec).
Fault = battery_starting_failure,
Rec = inspect_battery_and_starter.

?- diagnose(car3, Fault, Rec).
Fault = engine_mechanical_fault,
Rec = inspect_engine_internals.

?- diagnose(car4, Fault, Rec).
Fault = fuel_system_inefficiency,
Rec = replace_air_filter_service_fuel_system.

?- findall(V-F, (member(V,[car1,car2,car3,car4]), diagnose(V,F,_)), Results).
Results = [car1-cooling_system_failure, car2-battery_starting_failure,
          car3-engine_mechanical_fault, car4-fuel_system_inefficiency].
```

Because `diagnose/3` uses if-then-else (`->`) in a fixed priority order, Prolog's own SLD-resolution backward search does the work of both matching facts against rule bodies and backtracking across alternatives — the same mechanism that is demonstrated manually in Section 7.

6.3 Repository

The complete Prolog source, additional test vehicles and SWI-Prolog console transcripts for this problem are maintained in the course GitHub repository (see Section 10 for the link), alongside the earlier AT1 submission on the same automobile fault-diagnosis domain that this AT3 model builds upon.

7. Forward & Backward Chaining

Both reasoning strategies are demonstrated on the same working example: vehicle car1, with observed facts `coolant_level_low(car1)` and `temp_gauge_high(car1)`.

7.1 Forward Chaining (Data-Driven)

Forward chaining starts from the known facts and repeatedly fires any rule whose antecedent is fully satisfied, asserting new facts until no more rules can fire.

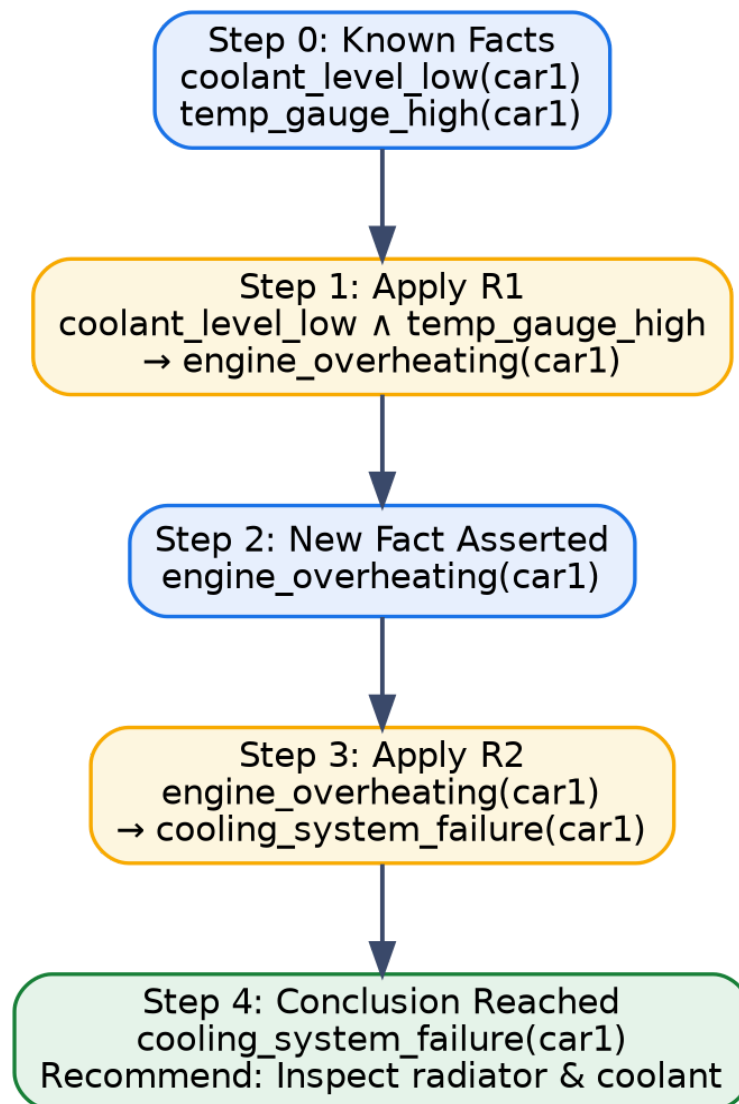


Figure 3: Forward-chaining trace from raw symptoms to the diagnosed fault for car1

7.2 Backward Chaining (Goal-Driven)

Backward chaining starts from a goal (a hypothesis to verify) and works backward, recursively reducing it to sub-goals until each sub-goal is matched against a known fact. Here the goal is: "Does car1 have `cooling_system_failure`?"

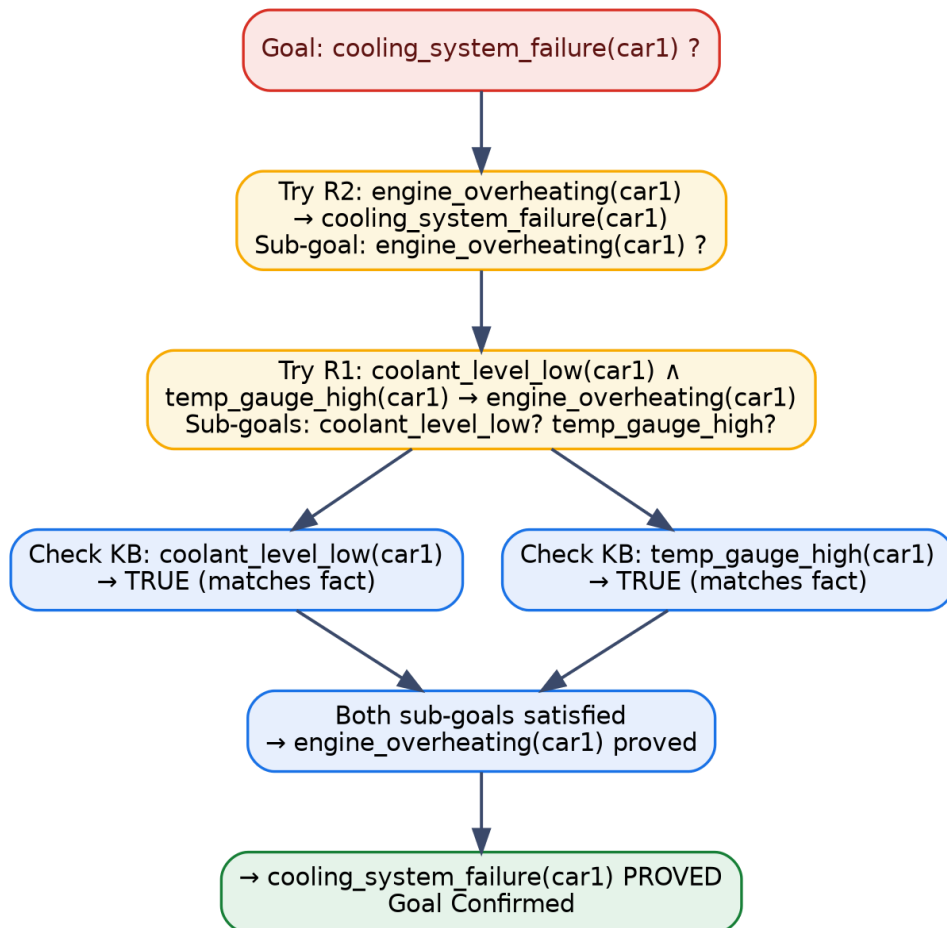


Figure 4: Backward-chaining proof tree for the goal `cooling_system_failure(car1)`

7.3 Comparison of the Two Traces

- Forward chaining is exhaustive and data-driven — useful when all symptoms are known upfront and every possible conclusion is wanted (e.g., a full vehicle health report).
- Backward chaining is goal-driven and typically more efficient when only a specific hypothesis needs verifying (e.g., a customer asks "is my overheating due to the cooling system?"), since it only explores facts relevant to that goal.
- Prolog's default resolution strategy is backward chaining, matching the trace in Figure 4 exactly; forward chaining must be simulated explicitly (e.g., using `assert/1` in a loop) since it is not Prolog's native mode.

8. Comparative Analysis of Modelling Approaches

The compulsory comparative table below evaluates all four models against the parameters specified in the assignment brief, followed by a radar-chart visualisation of their relative strengths.

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation	IF-THEN condition-action pairs	Atomic propositions + connectives	Predicates, variables, quantifiers	Facts & Horn-clause rules
Expressiveness	Moderate	Low	High	High
Rule Representation	Explicit, human-readable rules	Logical implications	Quantified formulas	head :- body clauses
Inference Mechanism	Pattern matching / rule firing	Modus ponens / resolution	Resolution + unification	SLD resolution + unification
Forward Chaining	Naturally supported	Possible, manual	Possible, less natural	Must be simulated (assert)
Backward Chaining	Supported (goal search)	Possible, manual proof	Naturally suited	Native default strategy
Handling Relationships	Weak (flat facts)	Very weak (no relations)	Strong (relations, variables)	Strong (unification)
Scalability	Moderate	Poor	Good	Good
Explainability	High (visible rule trace)	Moderate	Moderate (proof trees)	High (query trace)
Ease of Implementation	Easy (simple rule engine)	Easy	Moderate	Moderate (needs Prolog)
Suitability for the Problem	Good	Limited	Very good	Excellent

Table 5: Compulsory comparative table across the four modelling approaches

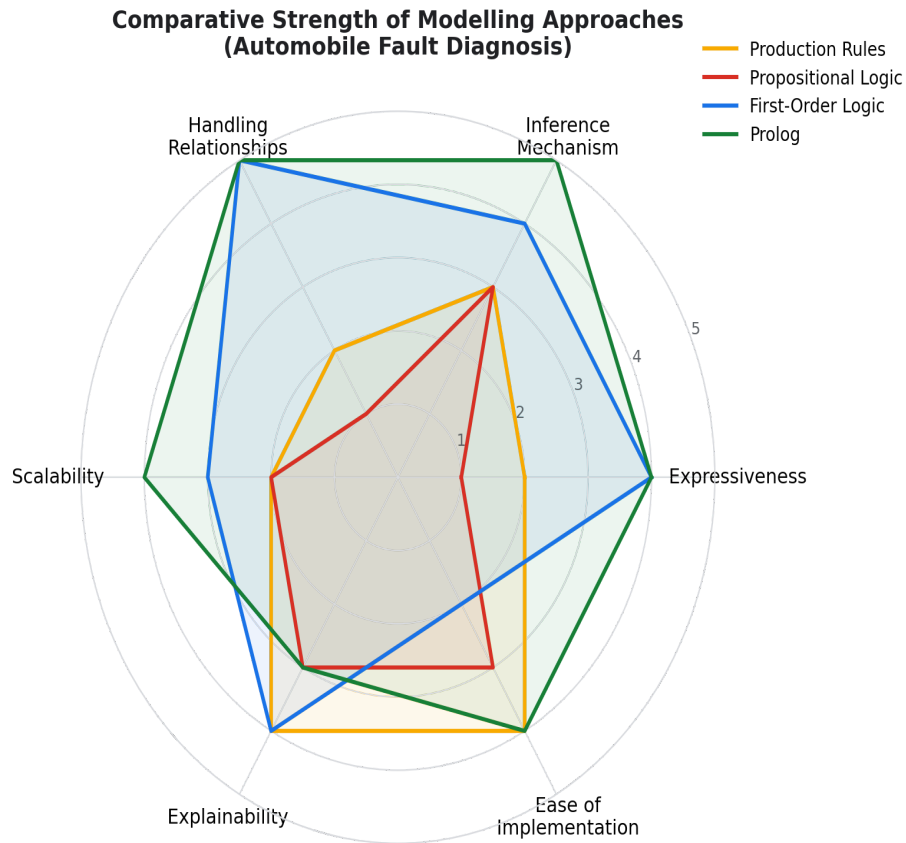


Figure 5: Relative strength of each modelling approach across six key parameters (1 = weak, 5 = strong)

8.1 Discussion

- Expressiveness & relationships: propositional logic is the weakest, since it cannot generalise across vehicles; FOL and Prolog are strongest because variables and unification let one rule apply to any vehicle.
- Inference & chaining: production rules and Prolog both support goal-driven (backward) reasoning naturally, but only production rules and (with simulation) Prolog support forward chaining cleanly; FOL's resolution can drive either style but needs an external control strategy.
- Scalability & explainability: production rules and Prolog scale well because rules/clauses are modular and can be added independently; production rules and Prolog also give the clearest, most human-readable trace of why a conclusion was reached.
- Ease of implementation: propositional logic and production rules are simplest to hand-build; Prolog needs an interpreter but then executes the logic directly, so no separate inference engine has to be written.

9. Results, Discussion and Recommended Model

All four models were validated on the same set of four sample vehicles (car1–car4) and produced identical diagnostic conclusions, confirming that the knowledge was translated consistently across representations: car1 → cooling_system_failure, car2 → battery_starting_failure, car3 → engine_mechanical_fault, and car4 → fuel_system_inefficiency.

9.1 Recommended Model

Prolog is recommended as the primary implementation approach for this automobile fault-diagnosis system. It combines the expressiveness and generality of first-order logic (variables, unification, one rule per fault class instead of one per vehicle) with an executable inference engine that requires no separate reasoning layer to be built. Its native backward-chaining behaviour matches how a service advisor typically interacts with the system — starting from a suspected fault and checking whether the evidence supports it — while forward-style behaviour can still be added for full vehicle health scans.

9.2 Role of the Other Models

- Production rules remain valuable as the authoring format: domain experts (mechanics) can review and edit IF–THEN rules without learning Prolog syntax, and these rules are then compiled into the Prolog clauses.
- Propositional logic is useful only as a teaching / verification aid for single-vehicle reasoning; it is not recommended for deployment due to its inability to generalise.
- First-order logic serves as the conceptual bridge between the human-readable production rules and the executable Prolog program, and is useful for formally proving the correctness of the rule set.

9.3 Limitations and Possible Improvements

- The current model is purely deterministic (Boolean); real diagnostic symptoms often carry uncertainty, so extending the rules with certainty factors or a fuzzy/probabilistic layer (e.g., Bayesian networks) would better reflect real-world ambiguity.
- The knowledge base is currently static; integrating live OBD-II sensor data feeds would let symptom facts be asserted automatically rather than entered manually.
- Conflict resolution between simultaneously firing rules (e.g., a vehicle with both overheating and starting failure) could be made more sophisticated with rule priorities or salience values.

10. Conclusion, References & GitHub Repository

10.1 Conclusion

This assignment modelled an automobile fault-diagnosis problem across four knowledge-representation paradigms of increasing expressive power — production rules, propositional logic, first-order logic, and Prolog — and demonstrated both forward and backward chaining over the resulting rule base. The comparative analysis shows a clear expressiveness/generality gradient, with Prolog offering the best overall balance of expressiveness, executability, explainability and ease of scaling to a full fleet of vehicles, making it the recommended approach for a deployable service-centre diagnostic tool, while production rules remain the most natural authoring format for domain experts.

10.2 References

1. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
2. Bratko, I. (2011). *Prolog Programming for Artificial Intelligence* (4th ed.). Addison-Wesley.
3. Giarratano, J. C., & Riley, G. (2005). *Expert Systems: Principles and Programming* (4th ed.). Course Technology.
4. Clocksin, W. F., & Mellish, C. S. (2003). *Programming in Prolog* (5th ed.). Springer.

5. SWI-Prolog Reference Manual. https://www.swi-prolog.org/pldoc/doc_for?object=manual

10.3 GitHub Repository

The Prolog source files, knowledge-base models, and test cases for this automobile fault-diagnosis assignment (building on the earlier AT1 submission for the same domain) are maintained at:

github.com/NikhilKumarReddyKadiveti/MLA0104-AIES

Note: the AT3 Prolog implementation, models, and test cases described in Section 6 should be committed to a corresponding "CO5 AT3" folder in the same repository, alongside the existing CO5 AT1 work, prior to final submission.

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear or poorly understood.
2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8% – Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.
3. Production Rule Modelling	15%	13–15% – Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12% – Rules are appropriate with minor errors or omissions.	7–9% – Basic rules are developed but several are incomplete or unclear.	0–6% – Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8% – Good propositional representation with minor errors.	5–6% – Basic representation with limited logical formulation.	0–4% – Representation is incorrect or incomplete.
5. First-Order Logic Modelling	15%	13–15% – Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL statements.	10–12% – Good FOL model with minor errors or omissions.	7–9% – Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6% – FOL model is largely incorrect or missing.
6. Prolog Modelling and Implementation	15%	13–15% – Successfully converts the knowledge model	10–12% – Functional Prolog implementa	7–9% – Basic implementation works but has limited	0–6% – Implementation is incomplete or non-functional.

		into Prolog facts, rules, predicates, and queries with correct outputs.	tion with minor technical issues.	functionality.	
7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8% – Both mechanisms are demonstrated with minor gaps.	5–6% – Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10% – Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8% – Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4% – Little or no meaningful comparison.
9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2% – Provides good analysis and reasonable justification.	1% – Basic discussion with limited justification.	0% – No meaningful analysis or justification.
10. Documentation, GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5% – Good documentation and presentation with minor issues.	1% – Basic documentation with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				